DOI:10.1145/2699417

Engineers use TLA+ to prevent serious but subtle bugs from reaching production.

BY CHRIS NEWCOMBE, TIM RATH, FAN ZHANG, BOGDAN MUNTEANU, MARC BROOKER, AND MICHAEL DEARDEUFF

How Amazon Web Services Uses Formal Methods

SINCE 2011, ENGINEERS at Amazon Web Services (AWS) have used formal specification and model checking to help solve difficult design problems in critical systems. Here, we describe our motivation and experience, what has worked well in our problem domain, and what has not. When discussing personal experience we refer to the authors by their initials.

At AWS we strive to build services that are simple for customers to use. External simplicity is built on a hidden substrate of complex distributed systems. Such complex internals are required to achieve high availability while running on cost-efficient infrastructure and cope with relentless business growth. As an example of this growth, in 2006, AWS launched S3, its Simple Storage Service. In the following six years, S3 grew to store one trillion objects.[3] Less than a year later it had grown to two trillion objects and was regularly handling 1.1 million requests per second.[4]

S3 is just one of many AWS services that store and process data our customers have entrusted to us. To safeguard that data, the core of each service relies on fault-tolerant distributed algorithms for replication, consistency, concurrency control, auto-scaling, load balancing, and other coordination tasks. There are many such algorithms in the literature, but combining them into a cohesive system is a challenge, as the algorithms must usually be modified to interact properly in a real-world system. In addition, we have found it necessary to invent algorithms of our own. We work hard to avoid unnecessary complexity, but the essential complexity of the task remains high.

Complexity increases the probability of human error in design, code, and operations. Errors in the core of the system could cause loss or corruption of data, or violate other interface contracts on which our customers depend. So, before launching a service, we need to reach extremely high confidence that the core of the system is correct. We have found the standard verification techniques in industry are necessary but not sufficient. We routinely use deep design reviews, code reviews, static code analysis, stress testing, and fault-injection testing but still find that subtle bugs can hide in complex concurrent fault-tolerant systems. One reason they do is that human intuition is poor at estimating the true probability of supposedly "extremely rare" combinations of events in systems operating at a scale of millions of requests per second.

» key insights

- **Formal methods find bugs in system designs that cannot be found through any other technique we know of.**
- **Formal methods are surprisingly feasible for mainstream software development and give good return on investment.**
- **At Amazon, formal methods are routinely applied to the design of complex real-world software, including public cloud services.**

IMAGE BY ANDRIJ BORYS ASSOCIATES

NASA's C. Michael Holloway says, "To a first approximation, we can say that accidents are almost always the result of incorrect estimates of the likelihood of one or more things."[8] Human fallibility means some of the more subtle, dangerous bugs turn out to be errors in design; the code faithfully implements the intended design, but the design fails to correctly handle a particular "rare" scenario. We have found that testing the code is inadequate as a method for finding subtle errors in design, as the number of reachable states of the code is astronomical. So we look for a better approach.

Precise Designs

In order to find subtle bugs in a system design, it is necessary to have a precise description of that design. There are at least two major benefits to writing a precise design: the author is forced to think more clearly, helping eliminate "plausible hand waving," and tools can be applied to check for errors in the design, even while it is being written. In contrast, conventional design documents consist of prose, static diagrams, and perhaps pseudo-code in an ad hoc untestable language. Such descriptions are far from precise; they are often ambiguous or missing critical aspects (such as partial failure or the granularity of concurrency). At the other end of the spectrum, the final executable code is unambiguous but contains an overwhelming amount of detail. We had to be able to capture the essence of a design in a few hundred lines of precise description. As our designs are unavoidably complex, we needed a highly expressive language, far above the level of code, but with precise semantics. That expressivity must cover real-world concurrency and fault tolerance. And, as we wish to build services quickly, we wanted a language that is simple to learn and apply, avoiding esoteric concepts. We also very much wanted an existing ecosystem of tools. We were thus looking for an off-the-shelf method with high return on investment.

We found what we were looking for in TLA+,[11] a formal specification language based on simple discrete math, or basic set theory and predicates, with which all engineers are familiar. A TLA+ specification describes the set of all possible legal behaviors, or execution traces, of a system. We found it helpful that the same language is used to describe both the desired correctness properties of the system (the "what") and the design of the system (the "how"). In TLA+, correctness properties and system designs are just steps on a ladder of abstraction, with correctness properties occupying higher levels, systems designs and algorithms in the middle, and executable code and hardware at the lower levels. TLA+ is intended to make it as easy as possible to show a system design correctly implements the desired correctness properties, through either conventional mathematical reasoning or tools like the TLC model checker[9] that take a TLA+ specification and exhaustively checks the desired correctness properties across all possible execution traces. The ladder of abstraction also helps designers manage the complexity of real-world systems; designers may choose to describe the system at several "middle" levels of abstraction, with each lower level serving a different purpose (such as to understand the consequences of fin-

er-grain concurrency or more detailed behavior of a communication medium). The designer can then verify that each level is correct with respect to a higher level. The freedom to choose and adjust levels of abstraction makes TLA+ extremely flexible.

At first, the syntax and idioms of TLA+ are somewhat unfamiliar to programmers. Fortunately, TLA+ is accompanied by a second language called PlusCal that is closer to a C-style programming language but much more expressive, as it uses TLA+ for expressions and values. PlusCal is intended to be a direct replacement for pseudo-code. Several engineers at Amazon have found they are more productive using PlusCal than they are using TLA+. However, in other cases, the additional flexibility of plain TLA+ has been very useful. For many designs the choice is a matter of taste, as PlusCal is automatically translated to TLA+ with a single key press. PlusCal users do have to be familiar with TLA+ in order to write rich expressions and because it is often helpful to read the TLA+ translation to understand the precise semantics of a piece of code. Moreover, tools (such as the TLC model checker) work at the TLA+ level.

A precise, testable description of a system becomes a what-if tool for designs, analogous to how spreadsheets are a what-if tool for financial models.

Formal Methods for Real-World Systems

In industry, formal methods have a reputation for requiring a huge amount of training and effort to verify a tiny piece of relatively straightforward code, so the return on investment is justified only in safety-critical domains (such as medical systems and avionics). Our experience with TLA+ shows this perception to be wrong. At the time of this writing, Amazon engineers have used TLA+ on 10 large complex real-world systems. In each, TLA+ has added significant value, either finding subtle bugs we are sure we would not have found by other means, or giving us enough understanding and confidence to make aggressive performance optimizations without sacrificing correctness. Amazon now has seven teams using TLA+, with encouragement from senior management and technical leadership. Engineers from entry level to principal have been able to learn TLA+ from scratch and get useful results in two to three weeks, in some cases in their personal time on weekends and evenings, without further help or training.

In this article, we have not included snippets of specifications because their unfamiliar syntax can be off-putting to potential new users. We find that potential new users benefit from hearing about the value of formal methods in industry before tackling tutorials and examples. We refer readers to Lamport et al.[11] for tutorials, Lamport's Viewpoint on page 38 in this issue, and Lamport[13] for an example of a TLA+ specification from industry similar in size and complexity to some of the larger specifications at Amazon (see the table here). We find TLA+ to be effective in our problem domain, but there are many other formal specification languages and tools, some of which we describe later.

Side Benefit

TLA+ has been helping us shift to a better way of designing systems. Engineers naturally focus on designing the "happy case" for a system, or the processing path in which no errors occur. This is understandable, as the happy case is by far the most common case. That code path must solve the customer's problem, perform well, make efficient use of resources, and scale with the business—all significant challenges in their own right. When the design for the happy case is done, the engineer then tries to think of "what could go wrong" based on personal experience and that of colleagues and reviewers. The engineer then adds mitigations for these scenarios, prioritized by intuition and perhaps statistics on the probability of occurrence. Almost always, the engineer stops well short of handling "extremely rare" combinations of events, as there are too many such scenarios to imagine.

In contrast, when using formal specification we begin by stating precisely "what needs to go right." We first specify what the system should do by defining correctness properties, which come in two varieties:

Safety. What the system is *allowed* to do. For example, at all times, all committed data is present and correct, or equivalently; at no time can the system have lost or corrupted any committed data; and

Liveness. What the system *must eventually* do. For example, whenever the

system receives a request, it must eventually respond to that request.

After defining correctness properties, we then precisely describe an abstract version of the design, along with an abstract version of its operating environment. We express "what must go right" by explicitly specifying all properties of the environment on which the system relies. Examples of such properties might be "If a communication channel has not failed, then messages will be propagated along it," and "If a process has not restarted, then it retains its local state, modulo any intentional modifications." Next, with the goal of confirming our design correctly handles all dynamic events in the environment, we specify the effects of each of those possible events—network errors and repairs, disk errors, process crashes and restarts, data-center failures and repairs, and actions by human operators. We then use the model checker to verify that the specification of the system in its environment implements the chosen correctness properties, despite any combination or interleaving of events in the operating environment. We find this rigorous "what needs to go right" approach to be significantly less error prone than the ad hoc "what might go wrong" approach.

More Side Benefits

We also find that writing a formal specification pays dividends over the lifetime of the system. All production services at Amazon are under constant development, even those released years ago; we add new features customers have requested, we redesign components to handle massive increases in scale, and we improve performance by removing bottlenecks. Many of these changes are complex and must be made to the running system with no downtime. Our first priority is always to avoid causing bugs in a production system, so we often have to answer "Is this change safe?" We find a major benefit of having a precise, testable model of the core system is that we can quickly verify that even deep changes are safe or learn they are unsafe without doing harm. In several cases, we have prevented subtle but serious bugs from reaching production. In other cases we have been able to make innovative performance optimizations (such as removing or narrowing locks or weakening constraints on message ordering) we would not have dared to do without having model-checked those changes. A precise, testable description of a system becomes a what-if tool for designs, analogous to how spreadsheets are a what-if tool for financial models. We find that using such a tool to explore the behavior of the system can improve the designer's understanding of the system.

In addition, a precise, testable, well-commented description of a design is an excellent form of documentation, which is important, as AWS systems have unbounded lifetimes. Over time, teams grow as the business grows, so we regularly have to bring new people up to speed on systems. This education must be effective. To avoid creating subtle bugs, we need all engineers to have the same mental model of the system and for that shared model to be accurate, precise, and complete. Engineers form mental models in various ways—talking to each other, reading design documents, reading code, and implementing bug fixes or small features. But talk and design documents can be ambiguous or incomplete, and the executable code is much too large to absorb quickly and might not precisely reflect the intended design. In contrast, a formal specification is precise, short, and can be explored and experimented on with tools.

What Formal Specification Is Not Good For

We are concerned with two major classes of problems with large distributed systems: bugs and operator errors that cause a departure from the system's logical intent; and surprising "sustained emergent performance degradation" of complex systems that inevitably contain feedback loops. We know how to use formal specification to find problems in the first class. However, problems in the second class can cripple a system even though no logic bug is involved. A common example is when a momentary slowdown in a server (due, perhaps, to Java garbage collection) causes timeouts to be breached on clients, causing the clients to retry requests, thus adding load to the server, and further slowdown. In such scenarios the system eventually makes progress; it is not stuck in a logical deadlock, livelock, or other cycle. But from the customer's perspective it is effectively unavailable due to sustained unacceptable response times. TLA+ can be used to specify an upper bound on response time, as a real-time safety property. However, AWS systems are built on infrastructure—disks, operating systems, network—that does not support hard real-time scheduling or guarantees, so real-time safety properties would not be realistic. We build soft real-time systems in which very short periods of slow responses are not considered errors. However, prolonged

Applying TLA+ to some of Amazon's more complex systems.

System	Components	Line Count (Excluding Comments)	Benefit
S3	Fault-tolerant, low-level network algorithm	804 PlusCal	Found two bugs, then others in proposed optimizations
	Background redistribution of data	645 PlusCal	Found one bug, then another in the first proposed fix
DynamoDB	Replication and group-membership system	939 TLA+	Found three bugs requiring traces of up to 35 steps
EBS	Volume management	102 PlusCal	Found three bugs
Internal distributed lock manager	Lock-free data structure	223 PlusCal	Improved confidence though failed to find a liveness bug, as liveness not checked
	Fault-tolerant replication-and-reconfiguration algorithm	318 TLA+	Found one bug and verified an aggressive optimization

severe slowdowns *are* considered errors. We do not yet know of a feasible way to model a real system that would enable tools to predict such emergent behavior. We use other techniques to mitigate these risks.

First Steps to Formal Methods

With hindsight, Amazon's path to formal methods seems straightforward; we had an engineering problem and found a solution. Reality was somewhat different. The effort began with author C.N.'s dissatisfaction with the quality of several distributed systems he had designed and reviewed, and with the development process and tools that had been used to construct those systems. The systems were considered successful, yet bugs and operational problems persisted. To mitigate the problems, the systems used well-proven methods—pervasive contract assertions enabled in production—to detect symptoms of bugs, and mechanisms (such as "recovery-oriented computing"[20]) to attempt to minimize the impact when bugs are triggered. However, reactive mechanisms cannot recover from the class of bugs that cause permanent damage to customer data; we must instead prevent such bugs from being created.

When looking for techniques to prevent bugs, C.N. did not initially consider formal methods, due to the pervasive view that they are suitable for only tiny problems and give very low return on investment. Overcoming the bias against formal methods required evidence they work on real-world systems. This evidence was provided by Zave,[22] who used a language called Alloy to find serious bugs in the membership protocol of a distributed system called Chord. Chord was designed by an expert group at MIT and is successful, having won a "10-year test of time" award at the SIGCOMM 2011 conference and influenced several systems in industry. Zave's success motivated C.N. to perform an evaluation of Alloy by writing and model checking a moderately large Alloy specification of a non-trivial concurrent algorithm.[18] We liked many characteristics of the Alloy language, including its emphasis on "execution traces" of abstract system states composed of sets and relations. However, we also found that Alloy is not expressive enough for many use cases at AWS; for instance, we could not find a practical way in Alloy to represent rich data structures (such as dynamic sequences containing nested records with multiple fields).

Alloy's limited expressivity appears to be a consequence of the particular approach to analysis taken by the Alloy Analyzer tool. The limitations do not seem to be caused by Alloy's conceptual model ("execution traces" over system states). This hypothesis motivated C.N. to look for a language with a similar conceptual model but with richer constructs for describing system states. C.N. eventually stumbled on a language with those properties when he found a TLA+ specification in the appendix of a paper on a canonical algorithm in our problem domain—the Paxos consensus algorithm.[12]

The fact that TLA+ was created by the designer of such a widely used algorithm gave us some confidence that TLA+ would work for real-world systems. We became more confident when we learned a team of engineers at DEC/Compaq had used TLA+ to specify and verify some intricate cache-coherency protocols for the Alpha series of multicore CPUs.[5,16] We read one of the specifications[13] and found they were sophisticated distributed algorithms involving rich message passing, fine-grain concurrency, and complex correctness properties. That left only the question of whether TLA+ could handle real-world failure modes. (The Alpha cache-coherency algorithm does not consider failure.) We knew from Lamport's Fast Paxos paper[12] that TLA+ could model fault tolerance at a high level of abstraction and were further convinced when we found other papers showing TLA+ could model lower-level failures.[15]

C.N. evaluated TLA+ by writing a specification of the same non-trivial concurrent algorithm he had written in Alloy.[18] Both Alloy and TLA+ were able to handle the problem, but the comparison revealed that TLA+ is much more expressive than Alloy. This difference is important in practice; several of the real-world specifications we have written in TLA+ would have been infeasible in Alloy. We initially had the opposite concern about TLA+; it is so expressive that no model checker can hope to evaluate everything that can be expressed in the language. But so far we have always been able to find a way to express our intent in a way that is clear, direct, and can be model checked.

After evaluating Alloy and TLA+, C.N. tried to persuade colleagues at Amazon to adopt TLA+. However, engineers have almost no spare time for such things, unless compelled by need. Fortunately, a need was about to arise.

First Big Success at Amazon

In January 2012, Amazon launched DynamoDB, a scalable high-performance "no SQL" data store that replicates customer data across multiple data centers while promising strong consistency.[2] This combination of requirements leads to a large, complex system.

The replication and fault-tolerance mechanisms in DynamoDB were created by author T.R. To verify correctness of the production code, T.R. performed extensive fault-injection testing using a simulated network layer to control message loss, duplication, and reordering. The system was also stress tested for long periods on real hardware under many different workloads. We know such testing is absolutely necessary but can still fail to uncover subtle flaws in design. To verify the design of DynamoDB, T.R. wrote detailed informal proofs of correctness that did indeed find several bugs in early versions of the design. However, we have also learned that conventional informal proofs can miss very subtle problems.[14] To achieve the highest level of confidence in the design, T.R. chose TLA+.

T.R. learned TLA+ and wrote a detailed specification of these components in a couple of weeks. To model-check the specification, we used the distributed version of the TLC model checker running on a cluster of 10 cc1.4xlarge EC2 instances, each with eight cores plus hyperthreads and 23GB of RAM. The model checker verified that a small, complicated part of the algorithm worked as expected for a sufficiently large instance of the system to give high confidence it is correct. T.R. then checked the broader fault-tolerant algorithm. This time the model checker found a bug that could lead to losing data if a particular sequence of failures and recovery steps would be interleaved with other processing. This was a very subtle bug; the

shortest error trace exhibiting the bug included 35 high-level steps. The improbability of such compound events is not a defense against such bugs; historically, AWS engineers have observed many combinations of events at least as complicated as those that could trigger this bug. The bug had passed unnoticed through extensive design reviews, code reviews, and testing, and T.R. is convinced we would not have found it by doing more work in those conventional areas. The model checker later found two bugs in other algorithms, both serious and subtle. T.R. fixed all these bugs, and the model checker verified the resulting algorithms to a very high degree of confidence.

T.R. says that, had he known about TLA+ before starting work on DynamoDB he would have used it from the start. He believes the investment he made in writing and checking the formal TLA+ specifications was more reliable and less time consuming than the work he put into writing and checking his informal proofs. Using TLA+ in place of traditional proof writing would thus likely have improved time to market, in addition to achieving greater confidence in the system's correctness.

After DynamoDB was launched, T.R. worked on a new feature to allow data to be migrated between data centers. As he already had the specification for the existing replication algorithm, T.R. was able to quickly incorporate this new feature into the specification. The model checker found the initial design would have introduced a subtle bug, but it was easy to fix, and the model checker verified the resulting algorithm to the necessary level of confidence. T.R. continues to use TLA+ and model checking to verify changes to the design for both optimizations and new features.

Formal methods have helped us devise aggressive optimizations to complex algorithms without sacrificing quality.

Persuading More Engineers

Success with DynamoDB gave us enough evidence to present TLA+ to the broader engineering community at Amazon. This raised a challenge—how to convey the purpose and benefits of formal methods to an audience of software engineers. Engineers think in terms of debugging rather than "verification," so we called the presentation "Debugging Designs."[18] Continuing the metaphor, we have found that software engineers more readily grasp the concept and practical value of TLA+ if we dub it "exhaustively testable pseudo-code." We initially avoid the words "formal," "verification," and "proof" due to the widespread view that formal methods are impractical. We also initially avoid mentioning what TLA stands for, as doing so would give an incorrect impression of complexity.

Immediately after seeing the presentation, a team working on S3 asked for help using TLA+ to verify a new fault-tolerant network algorithm. The documentation for the algorithm consisted of many large, complicated state-machine diagrams. To check the state machine, the team had been considering writing a Java program to brute-force explore possible executions: essentially a hard-wired form of model checking. They were able to avoid the effort by using TLA+ instead. Author F.Z. wrote two versions of the spec over a couple of weeks. For this particular problem, F.Z. found that she was more productive in PlusCal than TLA+, and we have observed that engineers often find it easier to begin with PlusCal.

Model checking revealed two subtle bugs in the algorithm and allowed F.Z. to verify fixes for both. F.Z. then used the spec to experiment with the design, adding new features and optimizations. The model checker quickly revealed that some of these changes would have introduced bugs.

This success led AWS management to advocate TLA+ to other teams working on S3. Engineers from those teams wrote specs for two additional critical algorithms and for one new feature. F.Z. helped teach them how to write their first specs. We find it encouraging that TLA+ can be taught by engineers who are still new to it themselves; this is important for quickly scaling adoption in an organization as large as Amazon.

Author B.M. was one such engineer. His first spec was for an algorithm known to contain a subtle bug. The bug had passed unnoticed through multiple design reviews and code reviews and had surfaced only after months of testing. B.M. spent two weeks learning TLA+ and writing the spec. Using it, the TLC model checker found the bug in seconds. The team had already designed and reviewed a fix for the bug,

so B.M. changed the spec to include the proposed fix. The model checker found the problem still occurred in a different execution trace. A stronger fix was proposed, and the model checker verified the second fix. B.M. later wrote another spec for a different algorithm. That spec did not uncover any bugs but did uncover several important ambiguities in the documentation for the algorithm the spec helped resolve.

Somewhat independently, after seeing internal presentations about TLA+, authors M.B and M.D. taught themselves PlusCal and TLA+ and started using them on their respective projects without further persuasion or assistance. M.B. used PlusCal to find three bugs and wrote a public blog about his personal experiments with TLA+ outside of Amazon.[7] M.D. used PlusCal to check a lock-free concurrent algorithm and then used TLA+ to find a critical bug in one of AWS's most important new distributed algorithms. M.D. also developed a fix for the bug and verified the fix. Independently, C.N. wrote a spec for the same algorithm that was quite different in style from the spec written by M.D., but both found the same bug in the algorithm. This suggests the benefits of using TLA+ are quite robust to variations among engineers. Both specs were later used to verify that a crucial optimization to the algorithm did not introduce any bugs.

Engineers at Amazon continue to use TLA+, adopting the practice of first writing a conventional prose-design document, then incrementally refining parts of it into PlusCal or TLA+. This method often yields important insight about the design, even without going as far as full specification or model checking. In one case, C.N. refined a prose design of a fault-tolerant replication system that had been designed by another Amazon engineer. C.N. wrote and model checked specifications at two levels of concurrency; these specifications helped him understand the design well enough to propose a major protocol optimization that radically reduced write-latency in the system. We have also discovered that TLA+ is an excellent tool for data modeling, as when designing the schema for a relational or "no SQL" database. We used TLA+ to design a non-trivial schema with semantic invariants over the data that were much richer than standard multiplicity constraints and foreign key constraints. We then added high-level specifications of some of the main operations on the data that helped us correct and refine the schema. This result suggests a data model can be viewed as just another level of abstraction of the entire system. It also suggests TLA+ may help designers improve a system's scalability. In order to remove scalability bottlenecks, designers often break atomic transactions into finer-grain operations chained together through asynchronous workflows; TLA+ can help explore the consequences of such changes with respect to isolation and consistency.

Executive management actively encourages teams to write TLA+ specs for new features and other significant design changes.

Most Frequently Asked Question

On learning about TLA+, engineers usually ask, "How do we know that the executable code correctly implements the verified design?" The answer is we do not know. Despite this, formal methods still help in multiple ways:

Get design right. Formal methods help engineers get the design right, which is a necessary first step toward getting the code right. If the design is broken, then the code is almost certainly broken, as mistakes during coding are extremely unlikely to compensate for mistakes in design. Worse, engineers are likely to be deceived into believing the code is "correct" because it appears to correctly implement the (broken) design. Engineers are unlikely to realize the design is incorrect while focused on coding;

Gain better understanding. Formal methods help engineers gain a better understanding of the design. Improved understanding can only increase the chances they will get the code right; and

Write better code. Formal methods can help engineers write better "self-diagnosing code" in the form of assertions. Independent evidence[10] and our own experience suggest pervasive use of assertions is a good way to reduce errors in code. An assertion checks a small, local part of an overall system invariant. A good system invariant captures the fundamental reason the system works; the system will not do anything wrong that could violate a safety property as long as it continuously maintains the system invariant.

The challenge is to find a good system invariant, one strong enough to ensure no safety properties are violated. Formal methods help engineers find strong invariants, so formal methods can help improve assertions that help improve the quality of code.

While we would like to verify that executable code correctly implements the high-level specification or even generate the code from the specification, we are not aware of any such tools that can handle distributed systems as large and complex as those being built at Amazon. We do routinely use conventional static analysis tools, but they are largely limited to finding "local" issues in the code, and are unable to verify compliance with a high-level specification.

We have seen research on using the TLC model checker to find "edge cases" in the design on which to test the code,[21] an approach that seems promising. However, Tasiran et al.[21] covered hardware design, and we have not yet tried to apply the method to software.

Alternatives to TLA+

There are many formal specification methods. We evaluated several and published our findings in Newcombe,[19] listing the requirements we think are important for a formal method to be successful in our industry segment. When we found TLA+ met those requirements, we stopped evaluating methods, as our goal was always practical engineering rather than an exhaustive survey.

Related Work

We find relatively little published literature on using high-level formal specification for verifying the design of complex distributed systems in industry. The Farsite project[6] is complex but somewhat different from the types of systems we describe here and apparently never launched commercially. Abrial[1] cited applications in commercial safety-critical control systems, but they seem less complex than our problem domain. Lu et al.[17] described post-facto verification of a well-known algorithm for a fault-tolerant distributed hash table, and Zave[22] described another such algorithm, but we do not know if these algorithms have been used in commercial products.

Conclusion

Formal methods are a big success at AWS, helping us prevent subtle but serious bugs from reaching production, bugs we would not have found through any other technique. They have helped us devise aggressive optimizations to complex algorithms without sacrificing quality. At the time of this writing, seven Amazon teams have used TLA+, all finding value in doing so, and more Amazon teams are starting to use it. Using TLA+ will improve both time-to-market and quality of our systems. Executive management actively encourages teams to write TLA+ specs for new features and other significant design changes. In annual planning, managers now allocate engineering time to TLA+.

While our results are encouraging, some important caveats remain. Formal methods deal with models of systems, not the systems themselves, so the adage "All models are wrong, some are useful" applies. The designer must ensure the model captures the significant aspects of the real system. Achieving it is a special skill, the acquisition of which requires thoughtful practice. Also, we were solely concerned with obtaining practical benefits in our particular problem domain and have not attempted a comprehensive survey. Therefore, mileage may vary with other tools or in other problem domains. C

References

1. Abrial, J. Formal methods in industry: Achievements, problems, future. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China, 2006), 761–768.
2. Amazon.com. *Supported Operations in DynamoDB: Strongly Consistent Reads.* System documentation; http://docs.aws.amazon.com/amazondynamodb/latest/developerguide/APISummary.html
3. Barr, J. Amazon S3: The first trillion objects. Amazon Web Services Blog, June 2012; http://aws.typepad.com/aws/2012/06/amazon-s3-the-first-trillion-objects.html
4. Barr, J. Amazon S3: Two trillion objects, 1.1 million requests per second. Amazon Web Services Blog, Mar. 2013; http://aws.typepad.com/aws/2013/04/amazon-s3-two-trillion-objects-11-million-requests-second.html
5. Batson, B. and Lamport, L. High-level specifications: Lessons from industry. In *Formal Methods for Components and Objects, Lecture Notes in Computer Science Number 2852*, F.S. de Boer, M. Bonsangue, S. Graf, and W.-P. de Roever, Eds. Springer, 2003, 242–262.
6. Bolosky, W., Douceur, J., and Howell, J. The Farsite Project: A retrospective. *ACM SIGOPS Operating Systems Review: Systems Work at Microsoft Research* 41, 2 (Apr. 2007), 17–26.
7. Brooker, M. Exploring TLA+ with two-phase commit. Personal blog, Jan. 2013; http://brooker.co.za/blog/2013/01/20/two-phase.html
8. Holloway, C. Michael Why you should read accident reports. Presented at the *Software and Complex Electronic Hardware Standardization Conference* (Norfolk, VA, July 2005); http://klabs.org/richcontent/conferences/faa_nasa_2005/presentations/cmh-why-read-accident-reports.pdf
9. Joshi, R., Lamport, L. et al. Checking cache-coherence protocols with TLA+. *Formal Methods in System Design 22*, 2 (Mar, 2003) 125–131.
10. Kudrjavets, G., Nagappan, N., and Ball, T. Assessing the relationship between software assertions and code quality: An empirical investigation. In *Proceedings of the 17th International Symposium on Software Reliability Engineering* (Raleigh, NC, Nov. 2006), 204–212.
11. Lamport, L. The TLA Home Page; http://research.microsoft.com/en-us/um/people/lamport/tla/tla.html
12. Lamport, L. Fast Paxos. *Distributed Computing 19*, 2 (Oct. 2006), 79–103.
13. Lamport, L. The Wildfire Challenge Problem; http://research.microsoft.com/en-us/um/people/lamport/tla/wildfire-challenge.html
14. Lamport, L. Checking a multithreaded algorithm with +CAL. In *Distributed Computing: 20th International Conference*, S. Dolev, Ed. Springer-Verlag, 2006, 11–163.
15. Lamport, L. and Merz, S. Specifying and verifying fault-tolerant systems. In *Formal Techniques in Real-Time and Fault-Tolerant Systems, Lecture Notes in Computer Science, Number 863*, H. Langmaack, W.-P. de Roever, and J. Vytopil, Eds. Springer-Verlag, Sept. 1994, 41–76.
16. Lamport, L., Sharma, M., Tuttle, M., and Yu, Y. *The Wildfire Challenge Problem*. Jan. 2001; http://research.microsoft.com/en-us/um/people/lamport/pubs/wildfire-challenge.pdf
17. Lu, T., Merz, S., and Weidenbach, C. Towards verification of the Pastry Protocol using TLA+. In *Proceedings of Joint 13th IFIP WG 6.1 International Conference and 30th IFIP WG 6.1 International Conference Lecture Notes in Computer Science Volume 6722* (Reykjavik, Iceland, June 6–9). Springer-Verlag, 2011, 244 –258.
18. Newcombe, C. Debugging Designs. Presented at the *14th International Workshop on High-Performance Transaction Systems* (Monterey, CA, Oct. 2011); http://hpts.ws/papers/2011/sessions_2011/Debugging.pdf and associated specifications http://hpts.ws/papers/2011/sessions_2011/amazonbundle.tar.gz
19. Newcombe, C. Why Amazon chose TLA+. In *Proceedings of the Fourth International Conference Lecture Notes in Computer Science Volume 8477*, Y.A. Ameur and K.-D. Schewe, Eds. (Toulouse, France, June 2–6). Springer, 2014, 25–39.
20. Patterson, D., Fox, A. et al. The Berkeley/Stanford Recovery-Oriented Computing Project. University of California, Berkeley; http://roc.cs.berkeley.edu/
21. Tasiran, S., Yu, Y., Batson, B., and Kreider, S. Using formal specifications to monitor and guide simulation: Verifying the cache coherence engine of the Alpha 21364 microprocessor. In *Proceedings of the Third IEEE International Workshop on Microprocessor Test and Verification* (Austin, TX, June). IEEE Computer Society, 2002.
22. Zave, P. Using lightweight modeling to understand Chord. *ACM SIGCOMM Computer Communication Review 42*, 2 (Apr. 2012), 49–57.

Chris Newcombe (chris.newcombe@gmail.com) is an architect at Oracle, Seattle, WA, and was a principal engineer in the AWS database services group at Amazon.com, Seattle, WA, when this article was written.

Tim Rath (rath@amazon.com) is a principal engineer in the AWS database services group at Amazon.com, Seattle, WA.

Fan Zhang (fanxhang58@gmail.com) is a software engineer and technical product and program manager at Cyanogen, Seattle, WA, and was a software engineer for AWS S3 at Amazon.com, Seattle, WA, when this article was written.

Bogdan Munteanu (bogdanmunte@gmail.com) is a software engineer at Dropbox, and was a software engineer in the AWS S3 Engines group at Amazon.com, Seattle, WA, when this article was written.

Marc Brooker (mbrooker@amazon.com) is a principal engineer for AWS EC2 at Amazon.com, Seattle, WA.

Michael Deardeuff (mdearde@amazon.com) is a software engineer in the AWS database services group at Amazon.com, Seattle, WA.